

EC-341 Digital Systems Design

Experiment # 8

Implementation of Verilog Parameters

Objectives

- Verilog Parameters
- Examples
- Lab Tasks

1. Verilog Parameters

Parameters are Verilog constructs that allow a module to be reused with a different specification. For example, a 4-bit adder can be parameterized to accept a value for the number of bits and new parameter values can be passed in during module instantiation. So, an N-bit adder can become a 4-bit, 8-bit or 16-bit adder. They are like arguments to a function that are passed in during a function call.

```
1 parameter MSB = 7;           // MSB is a parameter with a constant value
2 parameter REAL = 4.5;       // REAL holds a real number
3
4 parameter FIFO_DEPTH = 256,
5       MAX_WIDTH = 32;       // Declares two parameters
6
7 parameter [7:0] f_const = 2'b3; // 2 bit value is converted to 8 bit
```

Parameters are basically constants and hence it is illegal to modify their value at runtime. It is illegal to re-declare a name that is already used by a net, variable or another parameter. There are two major types of parameters, *module* and *specify* and both accept a range specification. But they are normally made as wide as the value to be stored requires them to be and hence a range specification is not necessary.

1.1.1 Module parameters

Module parameters can be used to override parameter definitions within a module, and this makes the module have a different set of parameters at compile time. A parameter can be modified with the *defparam* statement or in the module instance statement. It is

a common practice to use uppercase letters in names for the parameter to make them instantly noticeable.

The module shown below uses parameters to specify the bus width, data width and the depth of FIFO within the design and can be overridden with new values when the module is instantiated or by using *defparam* statements.

```
1  // Verilog 1995 style port declaration
2  module design_ip ( addr,
3                     wdata,
4                     write,
5                     sel,
6                     rdata);
7
8     parameter BUS_WIDTH    = 32,
9               DATA_WIDTH  = 64,
10             FIFO_DEPTH    = 512;
11
12     input addr;
13     input wdata;
14     input write;
15     input sel;
16     output rdata;
17
18     wire [BUS_WIDTH-1:0] addr;
19     wire [DATA_WIDTH-1:0] wdata;
20     reg [DATA_WIDTH-1:0] rdata;
21
22     reg [7:0] fifo [FIFO_DEPTH];
23
24     // Design code goes here ...
25 endmodule
```

In the new ANSI style of Verilog port declaration, you may declare parameters as shown below.

```
1 module design_ip
2     #(parameter BUS_WIDTH=32,
3       parameter DATA_WIDTH=64) (
4
5       input [BUS_WIDTH-1:0] addr,
6       // Other port declarations
7   );
```

1.1.2 Overriding parameters

Parameters can be overridden with new values during module instantiation. The first part instantiates the module called *design_ip* by the name *d0* where new parameters are passed in within *#()*. The second part uses a Verilog construct called *defparam* to set the new parameter values. The first method is the most used way to pass new parameters in RTL designs. The second method is commonly used in testbench simulations to quickly update the design parameters without having to re-instantiate the module.

```
1 module tb;
2
3     // Module instantiation override
4     design_ip #(BUS_WIDTH = 64, DATA_WIDTH = 128) d0 ( [port list])
5
6     // Use of defparam to override
7     defparam d0.FIFO_DEPTH = 128;
8
9 endmodule
```

2. Example

Following are the examples of Verilog code for counters.

2.1 2-bit Up-counter

```

module counter(
    input clk, reset, output[1:0] counter
);
    reg [1:0]out;

    always @(posedge clk or posedge reset)
    begin
        if(reset)
            out <= 2'b00;
        else
            out <= out + 2'b01;
        end
        assign counter = out;
    endmodule

```

```

module upcounter_testbench();
    reg clk, reset;
    wire [1:0] counter;

    counter dut(clk, reset, counter);
    initial begin
        clk=0;
        forever #5 clk=~clk;
    end
    initial begin
        reset=0;
        #10;
        reset=1;
        #1
        reset=0;
    end
endmodule

```

2.1.1 Using Parameters

The module counter has two parameters N and DOWN declared to have a default value of 2 and 0 respectively. N controls the number of bits in the output effectively controlling the width of the counter. By default, it is a 2-bit counter. Parameter DOWN controls whether the counter should increment or decrement. By default, the counter will decrement because the parameter is set to 0.

```

module counter
    #(
        parameter N=2,
        parameter DOWN=0
    )
    (
        input clk, reset, output[N-1:0] counter
    );
    reg [1:0]out;

    always @(posedge clk or posedge reset)
    begin
        if(reset)
            begin
                out <= 2'b00;
            end
        else
            begin
                if(DOWN)
                    out <= out - 2'b01;
                else
                    out <= out + 2'b01;
            end
    end

```

```

    end
    assign counter = out;
endmodule

```

The module counter is instantiated with N as 2 even though it is not required because the default value is anyway 2. DOWN is not passed in during module instantiation and hence takes the default value of 0 making it an up-counter.

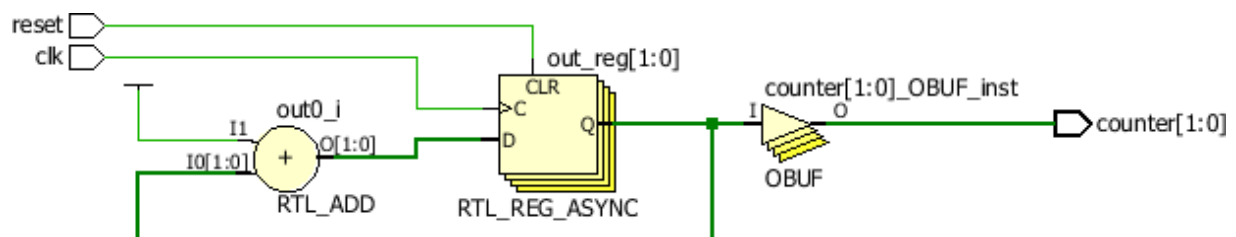
```

module counter_testbench();
    reg clk, reset;
    wire [1:0] counter;

    counter #(.N(2)) dut(clk, reset, counter);
    initial begin
        clk=0;
        forever #5 clk=~clk;
    end
    initial begin
        reset=0;
        #10;
        reset=1;
        #1
        reset=0;
    end
endmodule

```

See that default parameters are used to implement the counter where N equals two making it a 2-bit counter and DOWN equals zero making it an up-counter. The output from counter is left unconnected at the top level.



2.2 4-bit Down-counter

```

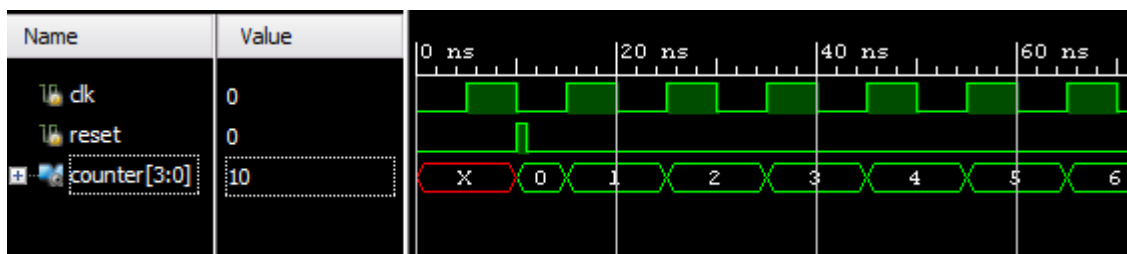
module counter(
    input clk, reset, output[3:0] counter
);
    reg [3:0]out;

    always @(posedge clk or posedge reset)
    begin
        if(reset)
            out <= 4'b0000;
        else
            out <= out + 4'b0001;
        end
        assign counter = out;
    endmodule

module counter_testbench();
    reg clk, reset;
    wire [3:0] counter;

    counter dut(clk, reset, counter);
    initial begin
        clk=0;
        forever #5 clk=~clk;
    end
    initial begin
        reset=0;
        #10;
        reset=1;
        #1
        reset=0;
    end
endmodule

```



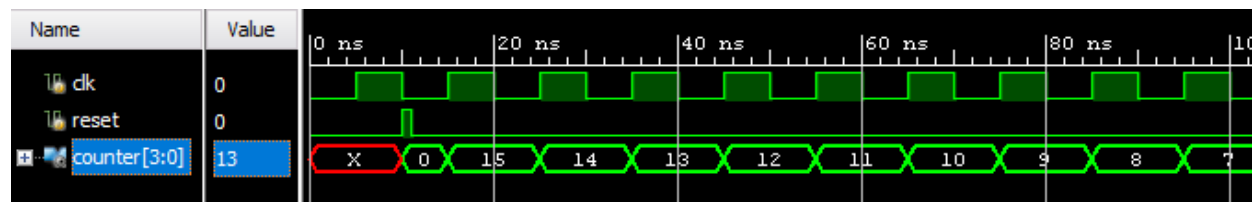
In this case, the module counter is instantiated with N as 4 making it a 4-bit counter. DOWN is passed a value of 1 during module instantiation and hence a down-counter is implemented.

```

module counter_testbench();
    reg clk, reset;
    wire [3:0] counter;

    counter #(.N(4),.DOWN(1)) dut(clk, reset, counter);
    initial begin
        clk=0;
        forever #5 clk=~clk;
    end
    initial begin
        reset=0;
        #10;
        reset=1;
        #1
        reset=0;
    end
endmodule

```



3. Tasks

1. Write a code for parameterized ALU with a parameter N which decides the operation to be performed by the ALU.
 - a. If N equals zero, ALU adds up the inputs.
 - b. If N equals one, ALU subtracts the inputs.
 - c. If N equals two, ALU compute the basic logic gates of the inputs.